# module6_lab2

October 11, 2026

## 1 Module 6 Lab 2: Build a Table from Scratch

**Name:** *Roey Kadur*
**Repo URL:** *https://github.com/roee0910/cs82a-portfolio.git*
**Problem used:** Appendix C, section C.2 worked example (CREATE TABLE and INSERT), adapted to SQLite. The appendix has no numbered Hands-On Problems.

Goal: create a `customers` table with a CREATE TABLE statement, populate it with INSERT, and prove the rows exist with SELECT.

### 1.1 Step 1: Connect to a new database

[1]:
```
import sqlite3
import pandas as pd

con = sqlite3.connect("lab2.db")

# Drop any old copy so the notebook can be re-run from the top (Restart & Run␣
 ↪All) without errors
con.execute("DROP TABLE IF EXISTS customers")
con.commit()
print("Connected to lab2.db")
```

```
Connected to lab2.db
```

### 1.2 Step 2: CREATE TABLE

Four columns: an integer id, two text columns (name, city) and one numeric column (total_spent).

[2]:
```
con.execute("""
CREATE TABLE customers (
    id          INTEGER PRIMARY KEY,
    name        TEXT NOT NULL,
    city        TEXT,
    total_spent REAL
)
""")
print("customers table created")
```

```
customers table created
```

### 1.3 Step 3: INSERT eight or more rows

All ten customers are fictional. Text values use straight single quotes.

[3]:
```
cur = con.execute("""
INSERT INTO customers (id, name, city, total_spent) VALUES
    (1,  'Maya Cohen',    'Tel Aviv',    1250.50),
    (2,  'Daniel Levi',   'Haifa',        830.00),
    (3,  'Noa Mizrahi',   'Jerusalem',   2145.75),
    (4,  'Yossi Katz',    'Tel Aviv',     415.20),
    (5,  'Tamar Peretz',  'Haifa',       1980.00),
    (6,  'Omer Biton',    'Beer Sheva',   260.90),
    (7,  'Shira Dahan',   'Jerusalem',    999.99),
    (8,  'Eitan Azulay',  'Tel Aviv',    3050.00),
    (9,  'Lior Friedman', 'Eilat',        120.00),
    (10, 'Hila Golan',    'Beer Sheva',   742.35)
""")

# Without commit() the inserts vanish when the connection closes
con.commit()
print(cur.rowcount, "rows inserted and committed")
```

```
10 rows inserted and committed
```

### 1.4 Step 4: Prove the rows exist with SELECT *

[4]:
```
pd.read_sql("SELECT * FROM customers", con)
```

[4]:

| | id | name | city | total_spent |
|---|---|---|---|---|
| 0 | 1 | Maya Cohen | Tel Aviv | 1250.50 |
| 1 | 2 | Daniel Levi | Haifa | 830.00 |
| 2 | 3 | Noa Mizrahi | Jerusalem | 2145.75 |
| 3 | 4 | Yossi Katz | Tel Aviv | 415.20 |
| 4 | 5 | Tamar Peretz | Haifa | 1980.00 |
| 5 | 6 | Omer Biton | Beer Sheva | 260.90 |
| 6 | 7 | Shira Dahan | Jerusalem | 999.99 |
| 7 | 8 | Eitan Azulay | Tel Aviv | 3050.00 |
| 8 | 9 | Lior Friedman | Eilat | 120.00 |
| 9 | 10 | Hila Golan | Beer Sheva | 742.35 |

#### 1.4.1 Check the structure and the row count

[5]:
```
schema = pd.read_sql("PRAGMA table_info(customers)", con)[["name", "type",␣
 ↪"pk"]]
print("Rows in customers:", con.execute("SELECT COUNT(*) FROM customers").
 ↪fetchone()[0])
schema
```

```
Rows in customers: 10
```

[5]:

| | name | type | pk |
|---|---|---|---|
| 0 | id | INTEGER | 1 |
| 1 | name | TEXT | 0 |
| 2 | city | TEXT | 0 |
| 3 | total_spent | REAL | 0 |

### 1.4.2 Prove the commit worked

A brand-new connection can only see data that was committed to the file.

[6]:
```
con_check = sqlite3.connect("lab2.db")
print("Rows seen by a second connection:", con_check.execute("SELECT COUNT(*)␣
 ↪FROM customers").fetchone()[0])
con_check.close()
```

```
Rows seen by a second connection: 10
```

### 1.4.3 Using the new table

The table behaves like any other: here is the Lab 1 pattern (GROUP BY) on it.

[7]:
```
pd.read_sql("""
    SELECT city,
           COUNT(*)         AS n_customers,
           SUM(total_spent) AS city_total
    FROM customers
    GROUP BY city
    ORDER BY city_total DESC
""", con)
```

[7]:

| | city | n_customers | city_total |
|---|---|---|---|
| 0 | Tel Aviv | 3 | 4715.70 |
| 1 | Jerusalem | 2 | 3145.74 |
| 2 | Haifa | 2 | 2810.00 |
| 3 | Beer Sheva | 2 | 1003.25 |
| 4 | Eilat | 1 | 120.00 |

## 1.5 MySQL to SQLite: the dialect difference I handled

Appendix C writes column types the MySQL way, such as `VARCHAR(50)` for text and `DECIMAL(10,2)` for money, and uses `AUTO_INCREMENT` for the id. SQLite has simpler type names, so I translated them: `VARCHAR(50)` became `TEXT`, `DECIMAL(10,2)` became `REAL`, and `INT AUTO_INCREMENT PRIMARY KEY` became `INTEGER PRIMARY KEY` (SQLite assigns the next id automatically for that column). I also retyped the book's curly quotes as straight quotes, since curly quotes break the SQL string in Python.

```
-- MySQL (book style)
CREATE TABLE customers (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50), city VARCHAR(50),
```

```
-- SQLite (this notebook)
CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT NOT NULL, city TEXT, total_spent REAL
```

### 1.6 Self-check

This cell fails loudly if the table does not meet the assignment requirements (at least 4 columns and at least 8 rows).

[8]:
```
n_cols = len(schema)
n_rows = con.execute("SELECT COUNT(*) FROM customers").fetchone()[0]
assert n_cols >= 4, "need at least 4 columns"
assert n_rows >= 8, "need at least 8 rows"
print(f"OK: {n_cols} columns, {n_rows} rows")
```

```
OK: 4 columns, 10 rows
```

[9]:
```
con.close()
print("Connection closed")
```

```
Connection closed
```